```
-- Step 0: Create and Use Database
CREATE DATABASE JOBORA;
USE JOBORA;
-- Step 1: Create the Users table
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    user_type VARCHAR(10) CHECK(user_type IN ('seeker', 'employer'))
);
-- Insert sample data into Users table
INSERT INTO Users (name, email, user_type) VALUES
('Ravi Kumar', 'ravi.kumar@mail.com', 'seeker'),
('Priya Sharma', 'priya.sharma@tcs.com', 'employer'),
('Neha Verma', 'neha.verma@mail.com', 'seeker'),
('Anil Mehta', 'anil.mehta@infosys.com', 'employer'),
('Rajesh Iyer', 'rajesh.iyer@mail.com', 'seeker');
-- Step 2: Create the JobSeekers table
CREATE TABLE JobSeekers (
    seeker_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
```

```
    skills VARCHAR(255),
    experience INT,
    preferred_location VARCHAR(100),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
-- Insert sample data into JobSeekers table
INSERT INTO JobSeekers (user_id, skills, experience, preferred_location) VALUES
(1, 'Python, SQL, Machine Learning', 1, 'Hyderabad'),
(3, 'HTML, CSS, JavaScript', 0, 'Remote'),
(5, 'Excel, Data Analysis, SQL', 2, 'Bangalore');
-- Step 3: Create the Employers table
CREATE TABLE Employers (
    employer_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    company_name VARCHAR(100),
    company_location VARCHAR(100),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
-- Insert sample data into Employers table
INSERT INTO Employers (user_id, company_name, company_location) VALUES
(2, 'TCS', 'Pune'),
```

```
(4, 'Infosys', 'Bangalore');

-- Step 4: Create the JobCategories table

CREATE TABLE JobCategories (

    category_id INT PRIMARY KEY AUTO_INCREMENT,

    category_name VARCHAR(100) UNIQUE

);

-- Insert data into JobCategories table

INSERT INTO JobCategories (category_name) VALUES

('Software'),

('Marketing'),

('HR'),

('Finance');


-- Step 5: Create the Jobs table

CREATE TABLE Jobs (

    job_id INT PRIMARY KEY AUTO_INCREMENT,

    employer_id INT,

    category_id INT,

    title VARCHAR(100),

    location VARCHAR(100),

    salary INT,
```

```
    required_skills VARCHAR(255),
    job_type VARCHAR(50),
    FOREIGN KEY (employer_id) REFERENCES Employers(employer_id),
    FOREIGN KEY (category_id) REFERENCES JobCategories(category_id)
);
-- Insert data into Jobs table
INSERT INTO Jobs (employer_id, category_id, title, location, salary, required_skills, job_type) VALUES
(1, 1, 'Data Analyst', 'Remote', 50000, 'SQL, Excel, Data Analysis', 'Remote'),
(2, 1, 'Frontend Developer', 'Chennai', 40000, 'HTML, CSS, JavaScript', 'Full-time'),
(1, 1, 'ML Developer', 'Hyderabad', 70000, 'Python, Machine Learning, SQL', 'Full-time');
-- Step 6: Create the Applications table
CREATE TABLE Applications (
    application_id INT PRIMARY KEY AUTO_INCREMENT,
    job_id INT,
    seeker_id INT,
    application_date DATE,
    status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (job_id) REFERENCES Jobs(job_id),
    FOREIGN KEY (seeker_id) REFERENCES JobSeekers(seeker_id)
);
```

```
-- Insert sample data into Applications table
INSERT INTO Applications (job_id, seeker_id, application_date, status) VALUES
(1, 1, CURDATE(), 'Pending'),
(3, 1, CURDATE(), 'Pending'),
(2, 3, CURDATE(), 'Pending');
-- Step 7: Create Stored Procedures
DELIMITER //

-- Procedure to apply for a job
CREATE PROCEDURE ApplyForJob(
    IN seekerID INT,
    IN jobID INT
)
BEGIN
    DECLARE already_applied INT;
    SELECT COUNT(*) INTO already_applied
    FROM Applications
    WHERE seeker_id = seekerID AND job_id = jobID;
    IF already_applied = 0 THEN
        INSERT INTO Applications (job_id, seeker_id, application_date, status)
        VALUES (jobID, seekerID, CURDATE(), 'Pending');
```

```
        SELECT 'Application submitted successfully.' AS Message;
    ELSE
        SELECT 'You have already applied for this job.' AS Message;
    END IF;
END //
-- Procedure to update the status of an application
CREATE PROCEDURE UpdateApplicationStatus(
    IN applicationID INT,
    IN newStatus VARCHAR(20)
)
BEGIN
    UPDATE Applications
    SET status = newStatus
    WHERE application_id = applicationID;
    SELECT 'Application status updated successfully.' AS Message;
END //
-- Procedure to search for jobs
CREATE PROCEDURE SearchJobs(
    IN keyword VARCHAR(100),
    IN city VARCHAR(100),
    IN skill VARCHAR(100)
```

```
)
BEGIN
    SELECT * FROM Jobs
    WHERE
        (keyword IS NULL OR title LIKE CONCAT('%', keyword, '%'))
        AND (city IS NULL OR location LIKE CONCAT('%', city, '%'))
        AND (skill IS NULL OR required_skills LIKE CONCAT('%', skill, '%'));
END //
DELIMITER ;
-- Step 8: Create the JobApplicationStatus View
CREATE VIEW JobApplicationStatus AS
SELECT
    a.application_id,
    j.title AS job_title,
    u.name AS seeker_name,
    a.application_date,
    a.status AS application_status
FROM
    Applications a
JOIN
    Jobs j ON a.job_id = j.job_id
```

```
JOIN
   JobSeekers js ON a.seeker_id = js.seeker_id
JOIN
   Users u ON js.user_id = u.user_id;
-- Step 9: Demonstration
-- Employer updates status
CALL UpdateApplicationStatus(1, 'Approved');
CALL UpdateApplicationStatus(2, 'Rejected');
-- Search for jobs
CALL SearchJobs('Java', 'Bangalore', NULL);
-- Insert new job and search again
INSERT INTO Jobs (employer_id, title, location, salary, required_skills, job_type,
category_id)
VALUES (2, 'Java Developer', 'Bangalore', 700000, 'Java, Spring', 'Full-Time', 1);
-- Apply for a job
CALL ApplyForJob(1, 2);
-- View status
SELECT * FROM JobApplicationStatus;
SELECT * FROM Applications WHERE job_id = 1;
SELECT * FROM Users;
SELECT * FROM Jobs;
```